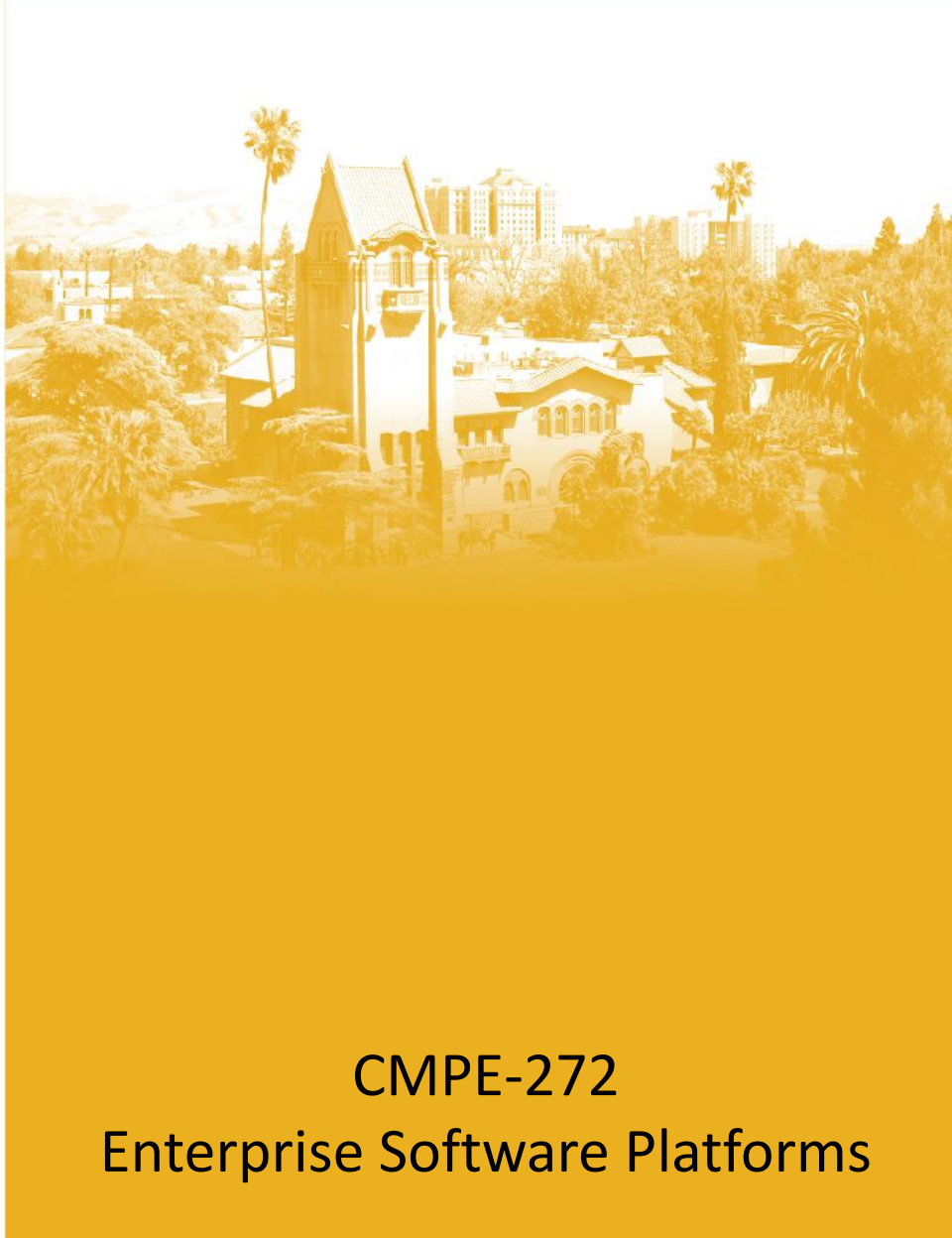




CMPE-272
Enterprise Software Platforms

Enterprise Software Platforms

Instructor: Andrew Bond

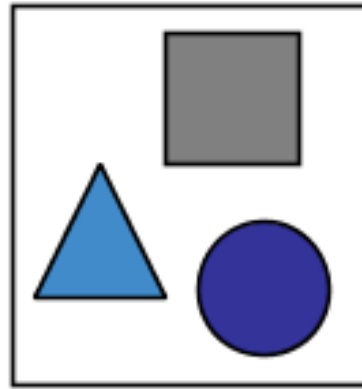
TIOBE Index for September 2025

Sep 2025	Sep 2024	Change	Programming Language	Ratings	Change
1	1		 Python	25.98%	+5.81%
2	2		 C++	8.80%	-1.94%
3	4	▲	 C	8.65%	-0.24%
4	3	▼	 Java	8.35%	-1.09%
5	5		 C#	6.38%	+0.30%
6	6		 JavaScript	3.22%	-0.70%
7	7		 Visual Basic	2.84%	+0.14%
8	8		 Go	2.32%	-0.03%
9	11	▲	 Delphi/Object Pascal	2.26%	+0.49%
10	27	▲▲	 Perl	2.03%	+1.33%
11	9	▼	 SQL	1.86%	-0.08%
12	10	▼	 Fortran	1.49%	-0.29%
13	15	▲	 R	1.43%	+0.23%

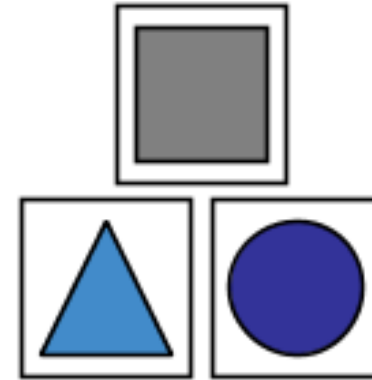
<https://www.tiobe.com/tiobe-index/>

UNIT 3 · CLOUD-NATIVE SERVICES

Monolith



Microservices

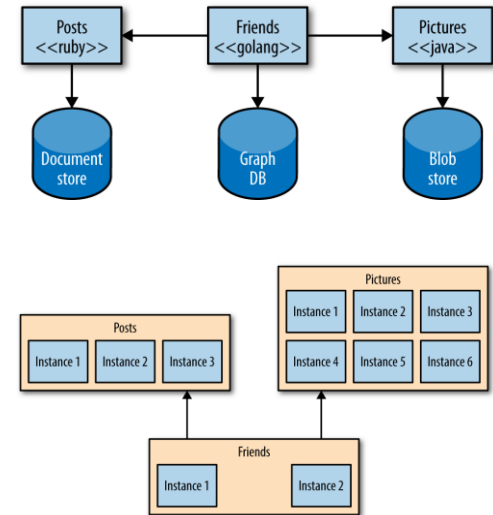


□ Process

MICROSERVICES

What Are Microservices?

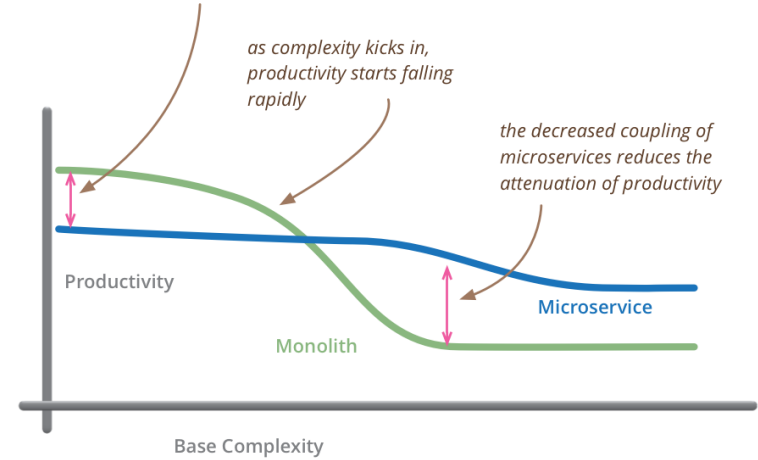
- **Small, and Focused on Doing One Thing Well**
Robert C. Martin's definition of the [Single Responsibility Principle](#), which states *"Gather together those things that change for the same reason, and separate those things that change for different reasons."*
- **Autonomous** - All communication between the services themselves are via network calls / APIs
- **Technology Heterogeneity** - pick the right tool for each job
- **Resilient**
- **Scalable**
- **Ease of Deployment**
- **Organizational Alignment**
- **Composability**
- **Optimized for Replaceability**



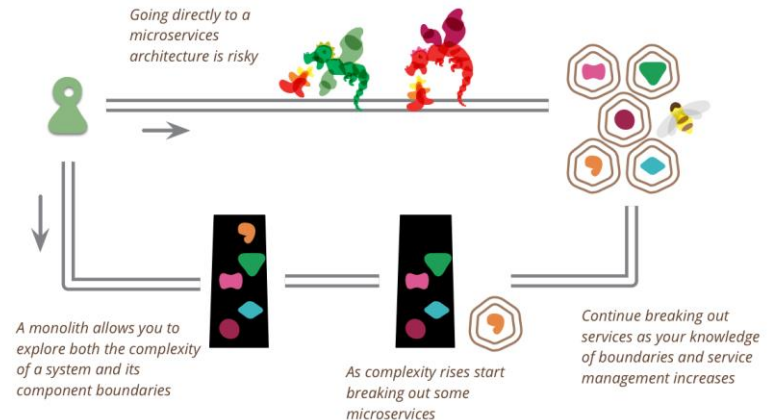
Why Microservices?

- Microservices are a useful architecture, but even their advocates say that using them incurs a significant [Microservice premium](#), which means they are only useful with more complex systems

for less-complex systems, the extra baggage required to manage microservices reduces productivity



but remember the skill of the team will outweigh any monolith/microservice choice



Architectural Style

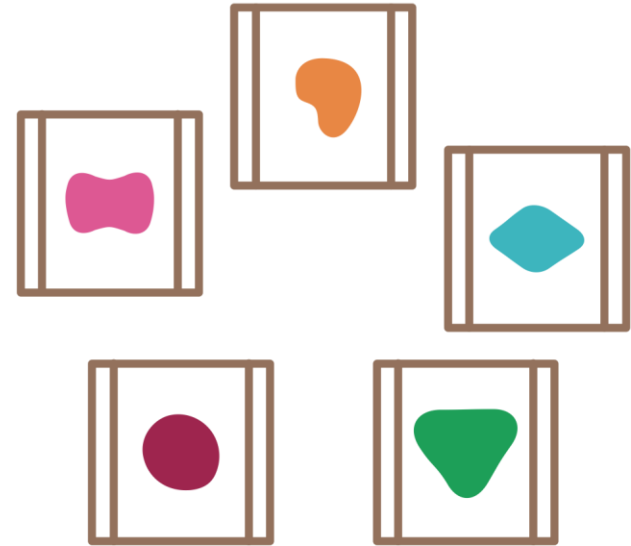
- a single application as a **suite of small services**, each **running in its own process** and communicating with lightweight mechanisms, often an HTTP resource API
- These services are **built around business capabilities** and **independently deployable** by fully automated deployment machinery
- There is a **bare minimum of centralized management** of these services, which may be written in different programming languages and use different data storage technologies

-- [James Lewis and Martin Fowler \(2014\)](#)

Microservices

common characteristics of this architectural style

by James Lewis and Martin Fowler



- Componentization via Services
- Organized around Business Capabilities
- Products not Projects
- Smart endpoints and dumb pipes
- Decentralized Governance
- Decentralized Data Management
- Infrastructure Automation
- Design for failure
- Evolutionary Design

2026 update: Modulith architecture is *a style of software design that emphasizes modularity within a monolithic application.*

Key Characteristics

1. **Decomposition:** An application is broken down into smaller, manageable services, each of which corresponds to a specific business capability.
2. **Independence:** Each microservice is independent, which means it can be developed, deployed, and scaled without affecting the operation of other services.
3. **Polyglot:** Given the independence, teams can choose the best programming languages, tools, and data storage technologies that fit the particular service's needs.
4. **Decentralized Governance:** Instead of a single monolithic architecture dictating the use of specific tools or technologies, microservices allow for a decentralized approach, where each service can potentially have its own tech stack.
5. **Communication:** Services communicate with each other often using lightweight protocols such as HTTP/REST, although other communication mechanisms like gRPC, message queues (like RabbitMQ or Kafka), or event-driven architectures can be used.
6. **Stateless:** Ideally, microservices should be stateless, meaning they shouldn't maintain any internal state between requests. If state is required, it should be outsourced to a database or caching mechanism.

Microservices provide benefits...

- [Strong Module Boundaries](#): Microservices reinforce modular structure, which is particularly important for larger teams.
- [Independent Deployment](#): Simple services are easier to deploy, and since they are autonomous, are less likely to cause system failures when they go wrong.
- [Technology Diversity](#): With microservices you can mix multiple languages, development frameworks and data-storage technologies.

...but come with costs

- [Distribution](#): Distributed systems are harder to program, since remote calls are slow and are always at risk of failure.
- [Eventual Consistency](#): Maintaining strong consistency is extremely difficult for a distributed system, which means everyone has to manage eventual consistency.
- [Operational Complexity](#): You need a mature operations team to manage lots of services, which are being redeployed regularly.

SLO in the age of Microservices

What are SLI, SLO and SLA anyway?

- **SRE** practice revolves around the concepts of **SLO**, **SLI**, **SLA**, and the related “**error budget**”. The **Google SRE Book**, which laid the foundations for the practice, defines them as follows:
- **Service Level Indicator (SLI)** is a carefully defined quantitative measure of some aspect of the level of service that is provided. Common examples include latency, error rate, request throughput and availability.
- **Service Level Objective (SLO)** is a target value or range of values for a service level that is measured by an SLI. For example, an SLO can state that the average latency per request should be under 120 milliseconds. Many companies set SLO targets as a number of nines, for example “five nines” uptime means 99.999% uptime, which means a maximum of 5.26 minutes downtime per year.
- **Service Level Agreement (SLA)** is an explicit or implicit contract with your users that includes consequences of meeting (or missing) the SLOs they contain. Put simply, if you’ve got a penalty attached to breaching an SLO – you’re talking SLA.
- **Error budget** is an important related concept, which determines the rate at which the SLOs can be missed. The Error budget is an important indicator for meeting the SLO goals consistently, and SRE teams track it weekly and even daily. An error budget is also important, as it enables downtime windows that can be used for maintenance, experimenting with new features, and other innovations.

<https://sre.google/sre-book/table-of-contents/>

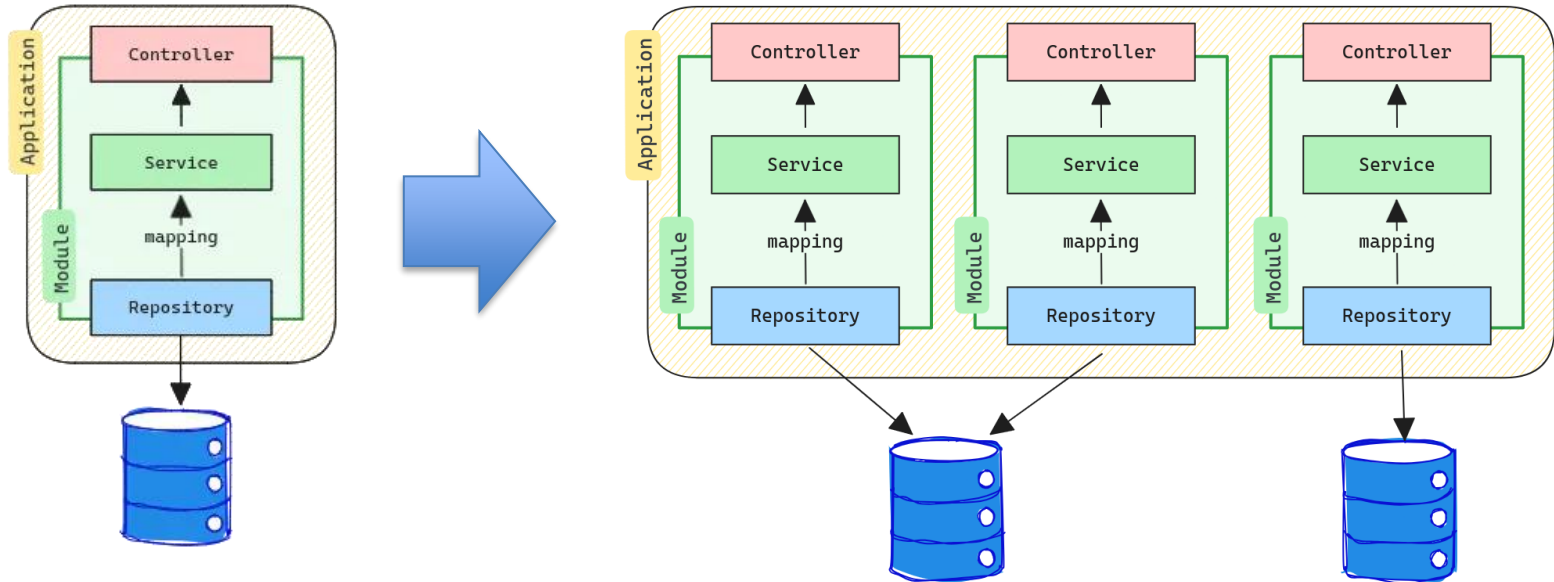
<https://logz.io/blog/sre-revisited-slo-in-the-age-of-microservices/>

Synchronous Versus Asynchronous Communication

- **Synchronous communication**, a call is made to a remote server, which blocks until the operation completes [Simpler to implement]
- **Asynchronous communication**, the caller doesn't wait for the operation to complete before returning, and may not even care whether or not the operation completes at all
- Different modes of communication can enable two different idiomatic styles of collaboration:
request/response or *event-based*

Architecture Style: Modulith (vs. Microservices)

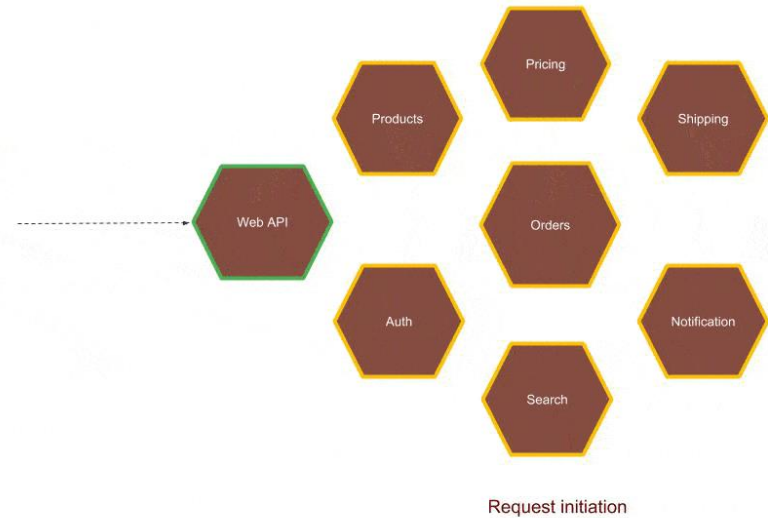
- **Modulith** architecture is a style of software design that emphasizes modularity within a monolithic application.
- It combines the simplicity and straightforward deployment model of a monolithic architecture with the modularity and maintainability typically associated with microservices.



SYNCHRONOUS AND ASYNCHRONOUS ARCHITECTURE PATTERNS

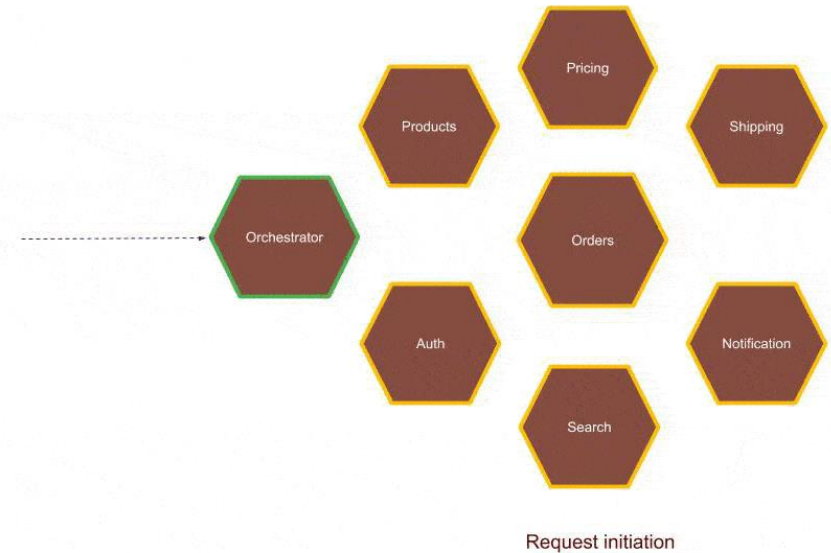
De-Centralized and Synchronous

- Intercepts a flow at the entry point
- Calls remain synchronous throughout the system
- Not well suited for a complex workflow that is susceptible to change
- Not ideal for a system with high read/write frequency



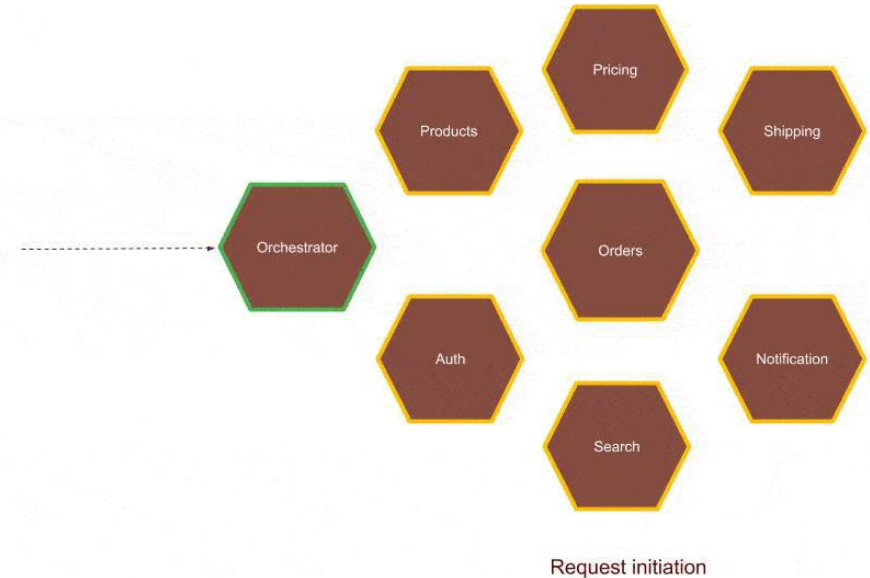
Orchestrated, Synchronous, and Sequential

- A variation of synchronous communication is with a central orchestrator
- Each service, in turn, responds back to the orchestrator.
- The orchestrator continues to hold all active requests. This burdens orchestrator more than other services.
- The orchestrator is susceptible to being a single point of failure
- This style of architecture is still suitable for a read-heavy system



Orchestrated, Synchronous, and Parallel

- Small improvement on the previous approach is to make independent requests parallel
- It can allow for faster execution of a flow. With shorter response times, orchestrator can have a higher throughput
- Workflow management is more complex than the previous approach
- Due to its synchronous nature, the system is still better for a read-heavy architecture



Synchronous Architecture Trade-offs

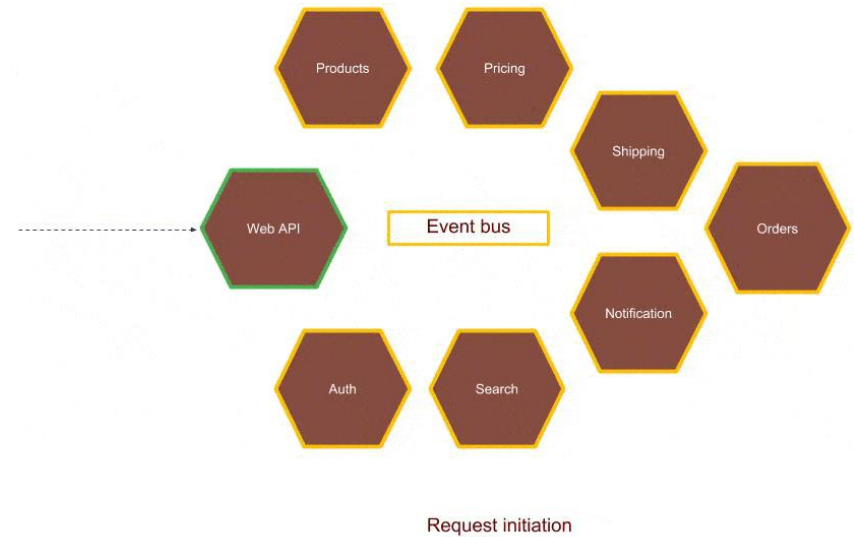
- Balanced Capacity
- Risk of Cascading Failures
- Increased Load Balancing & Service Discovery Overhead
- Coupling (increasing the system complexity, and brittleness)

Asynchronous

- Well suited for a distributed architecture.
- Direct calls to a remote service over RPC (for instance, gRPC) or via a mediating message bus are common implementations
- advantages of a central message bus is consistent communication and message delivery semantics

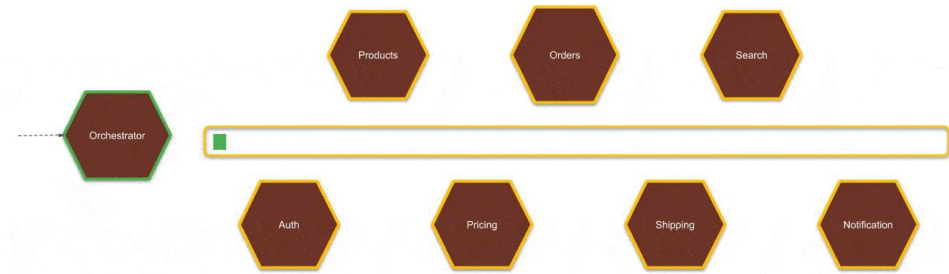
Choreographed Asynchronous Events

- each component listens to a central message bus and awaits an event
- Any context needed by execution is part of the event payload
- Triggering of downstream events is a responsibility that each service owns
- Scales well for a write-heavy system



Orchestrated, Asynchronous, and Sequential

- Each service is a producer and consumer to the central message bus
- Responsibilities of orchestrator involve routing messages to their corresponding services
- Each component consumes an incoming event or message and produces the response back on the message queue
- Orchestrator consumes this response and does transformation before routing ahead to next step



Hybrid With Orchestration and Event Choreography

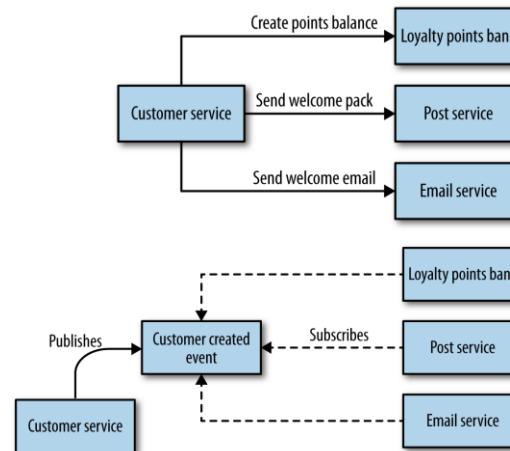
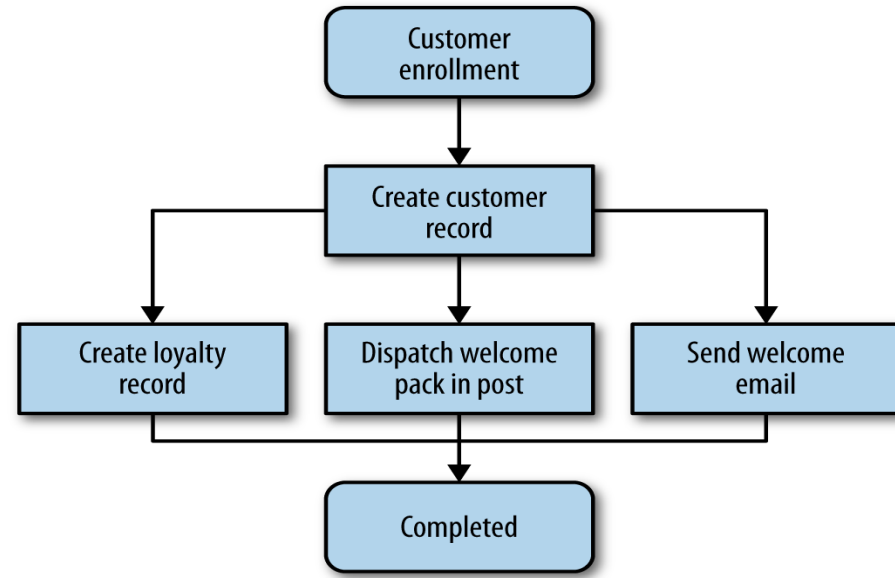
- The orchestration is excellent for explicit flow execution, while choreography can handle implicit execution
- This amalgamation of two approaches provides best of both worlds
- Workflow specification can facilitate emanation of events at specific steps
- Independent execution of tasks like notifications, indexing, et-cetera, while the orchestration can continue to drive explicit execution

Asynchronous Architecture Trade-offs

- Service flows that are asynchronous in nature can be hard to follow through the system
- Asynchronous systems tend to be significantly more complex than synchronous ones
- Asynchronous architecture is a natural fit for the write-heavy system. However, it needs mediation for synchronous reads/queries
- sync wrapper over an async system. This is an entry point that can invoke asynchronous flows downstream. A synchronous wrapper is a *stateful component*

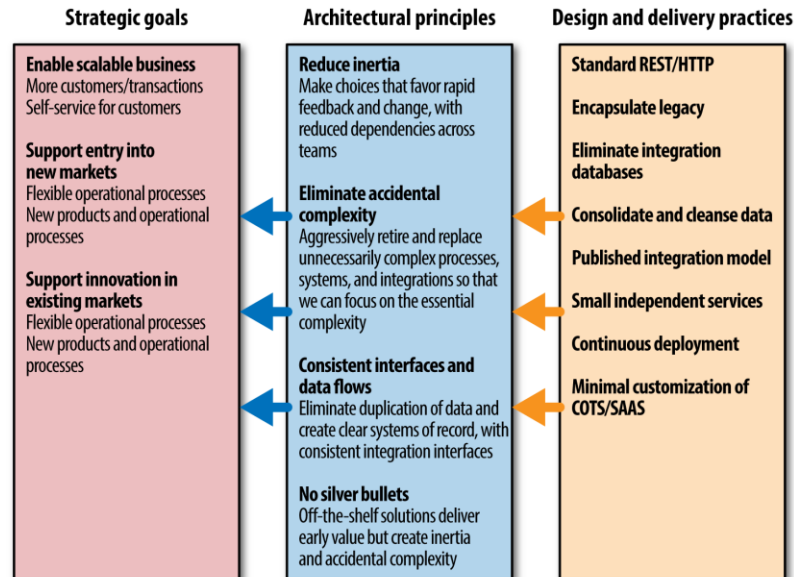
Orchestration Versus Choreography

- Take an example from MusicCorp, and look at what happens when we create a customer:
- A new record is created in the loyalty points bank for the customer.
- Our postal system sends out a welcome pack.
- We send a welcome email to the customer
- **Handling customer creation via orchestration**
- **Handling customer creation via choreography**



Architectural Considerations

- Worry less about what happens *inside* the service than what happens *between* the services
- System Design



Governance Through Code

Governance ensures that enterprise objectives are achieved by evaluating stakeholder needs, conditions and options; setting direction through prioritization and decision making; monitoring performance, compliance and progress against agreed-on direction and objectives.

- Exemplars
- Tailored Service Template
 - The template contains a default set of decisions such as web frameworks, logging, monitoring, build, packaging, and deployment approaches. This is a very useful technique for encouraging collaborative evolution while retaining lightweight governance
 - Need template for each stack used
 - be careful that creating the service template doesn't become the job of a central tools or architecture team

Modeling Services

- **Loose Coupling** (change to one service should not require a change to another)
- **High Cohesion** (related behavior together, unrelated behavior apart)

Service Boundary Checklist

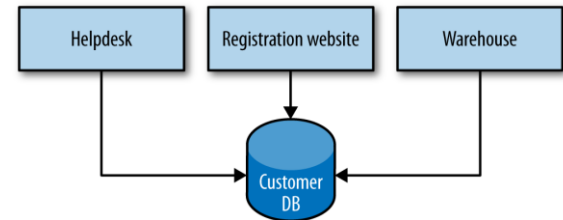
- capability name
- owned aggregates
- APIs/events
- Datastore
- SLOs
- Team
- deploy cadence
- dependencies (sync/async)

Top 3 failure modes?

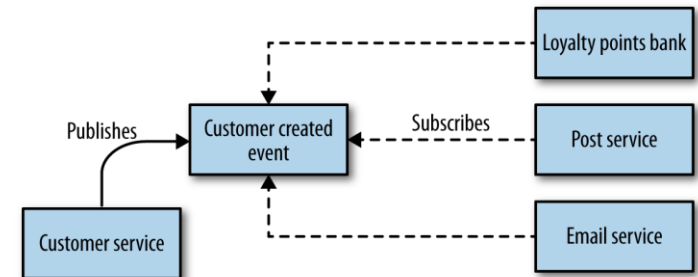
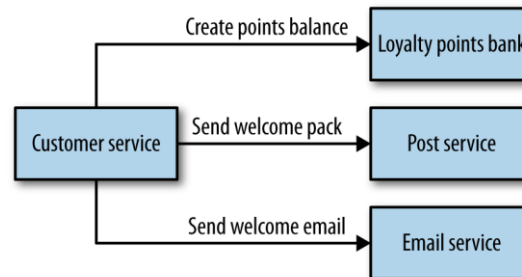
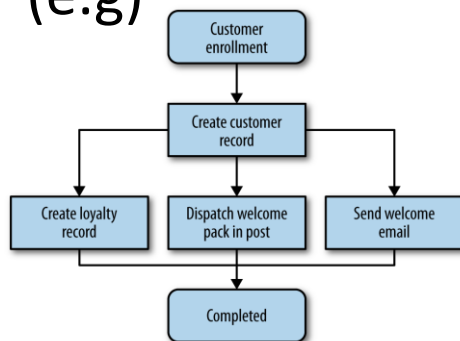
- **Dependency brownouts → timeouts → cascading failure**
 - **Symptoms:** rising p95/p99 latency, timeout rate spikes, thread/conn pools pinned, CBs opening.
 - **Guards:** strict per-hop timeouts, retry budgets with exponential backoff + jitter, circuit breakers + graceful fallbacks, small caches for hot reads.
 - **Monitors:** upstream latency/timeout %, CB open %, queueing delay.
- **Saturation / backpressure failure (resource exhaustion)**
 - **Symptoms:** CPU/mem/FD or DB connections pegged, queue/backlog growth, 429/503s, GC pauses. Often triggered by traffic spikes or retry storms.
 - **Guards:** bulkheads (per-dep pools/quotas), bounded queues & concurrency limits, load shedding/rate limiting, autoscale on queue depth/lag, isolate by cell/AZ.
 - **Monitors:** consumer lag, runnable threads, pool utilization, shed/limited request rate.
- **Contract & state correctness failures (schema drift, duplicates, ordering)**
 - **Symptoms:** parse/validation errors, silent truncation, 4xx bursts after a deploy, double-charges or “ghost” records from duplicate/reordered events.
 - **Guards:** versioned schemas (JSON/Proto) with backward compatibility, schema registry + contract tests, idempotency keys & unique constraints, outbox (“publish on commit”), sagas/compensations, reconciliation jobs.
 - **Monitors:** schema-validation failures, idempotency-conflict rate, outbox backlog, reconciliation diffs.

Integration

- Avoid Breaking Changes
- Keep APIs Technology-Agnostic
- Hide Internal Implementation Detail
- Avoid Shared Database (exceptions: read-only/strangler)
- Sync Vs. Async
 - [synchronization of algorithms](#)
 - [logical ordering](#)
 - [time synchronization](#)
- Customer Creation: Orchestration Versus Choreography



(e.g)



Sync vs Async matrix

Mode

When to Use

Pros

Cons

Sync (request/ response: HTTP/gRPC)

- User is waiting (UI/blocking flows)
- Need fresh data right now
- Small, bounded work
- Few downstream hops

- Simple mental model
- Immediate result/clear errors
- Easier debugging & tracing inline
- Good for reads/validations

- Tight coupling; upstream SLO = sum of downstream latencies
- Cascading failures if deps are slow
- Retry storms without care
- N+1 call patterns add tail latency

Async (events/ queues/ topics)

- Side-effects & fan-out (email, points, webhooks)
- Spike smoothing/backpressure needed
- Long-running or retrievable work
- Cross-team integration

- Decouples teams and deploys
- Buffers spikes; resilient with retries/DLQ
- Scales consumers independently
- Natural audit log of changes

- Eventual consistency
- Must handle duplicates/ordering (idempotency keys, keys-by-aggregate)
- Harder end-to-end visibility without strong observability
- More moving parts (schemas, DLQs, redrives)

Asynchronous Event-Based Collaboration

- Beware of Middleware (best practice - keep your middleware dumb, and keep the smarts in the endpoints)
- Prefer open contracts: HTTP + CloudEvents (webhooks/SSE/WebSocket), or a broker (Kafka, RabbitMQ, SQS/Pub/Sub). Version messages; document schemas.
- Event-driven architectures lead to decoupled, scalable systems
- See [Enterprise Integration Patterns](#) (Addison-Wesley)

Checklist

- **Contract:** CloudEvents + JSON/Protobuf
- **Delivery:** at-least-once + idempotent handlers
- **Reliability:** retries + DLQ + backoff
- **State:** outbox pattern for “publish on commit”
- **Observability:** trace + metrics + alarms

Producer → Topic/Queue → Consumers with DLQ and Schema Registry

Contract

1) **Contract:** CloudEvents + JSON/Protobuf

Use CloudEvents 1.0 attributes on every message: id, source, type, subject, time, datacontenttype, dataschema.

- **Payload (“data”) format:**

- JSON: easiest to debug; fine for moderate throughput.
- Protobuf (or Avro): smaller/faster, typed, great for high-volume links.

- **Versioning:**

- Version event “type” (e.g., com.shop.order.v2.created) and/or include data.schemaVersion.
- Backward-compatible changes only (add optional fields; never change meaning or remove without deprecation window).

- **Schema Registry:**

- Store JSON Schema/Protobuf/Avro with a COMPATIBILITY policy (BACKWARD or BACKWARD_TRANSITIVE).
- Validate on publish and/or on consume; reject/alert on violations.

Delivery

2) **Delivery**: at-least-once + idempotent handlers

Assume duplicates and reordering; “exactly-once” is an application effect, not a transport guarantee.

- **Idempotency keys:**

- Prefer CloudEvents “id” or a domain key (orderId + version).
- Enforce once-only effects via:
 - (a) Upsert/INSERT ... ON CONFLICT (unique key on idempotency key),
 - (b) “ProcessedEvents(id, timestamp)” table with TTL + unique index,
 - (c) Compare-and-set on an aggregate’s version (optimistic concurrency).

- **Ordering:**

- Design aggregates so each key is processed by one partition/ordering key (Kafka key, SQS FIFO group, Pub/Sub ordering key).

Reliability

3) **Reliability**: retries + DLQ + backoff

Retries:

- Exponential backoff with FULL JITTER (avoid thundering herds).
- Classify errors: transient (retry) vs permanent (no retry).

- **DLQ (Dead Letter Queue):**

- Send to DLQ after retry budget exhausted OR immediately on non-retryable errors.
- Include failure metadata: last exception, stack trace, attempt count, original headers.
- Operate DLQ: retention policy; periodic redrive with guardrails; dashboards.

- **Safety valves:**

- Concurrency limits; circuit breakers on sustained 5xx/timeout rates; poison-pill detection (same id fails repeatedly).

State

4) **State:** outbox pattern (“publish on commit”)

- Within the service’s DB transaction that mutates business state, also write an OUTBOX row:

```
Outbox(id uuid, aggregate_id, event_type,  
       payload, headers, status='NEW', created_at)
```

- A relay (poller) or CDC stream (e.g., Debezium) publishes NEW rows to the broker; mark status='SENT' (idempotent: publisher stores message id).
- **Guarantees:**
 - No lost events (commit == event recorded).
 - No phantom events (no publish without commit).
- Correlate the broker key with aggregate_id for stable partitioning.

Observability

5) **Observability:** trace + metrics + alarms

- **Tracing:**

- Propagate W3C traceparent (and baggage) across producer → broker → consumer.
- Add a business correlationId (e.g., orderId) to logs/metrics.

- **Metrics to ship:**

- Publish rate, consume rate, end-to-end latency (p50/p95/p99), consumer lag/backlog, retry counts, DLQ depth, schema-validation failures.

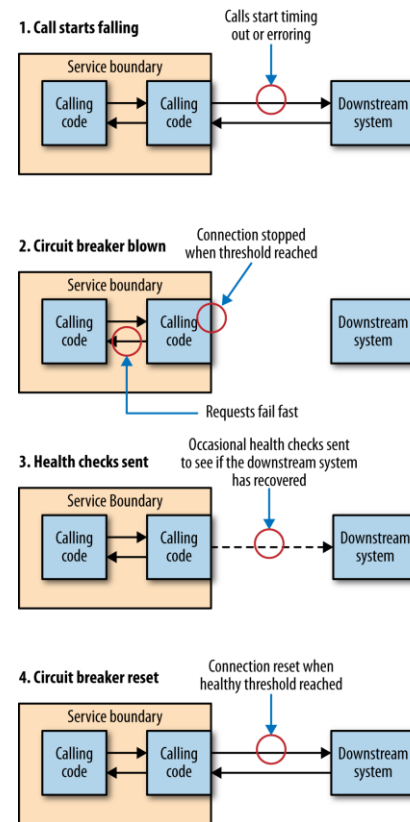
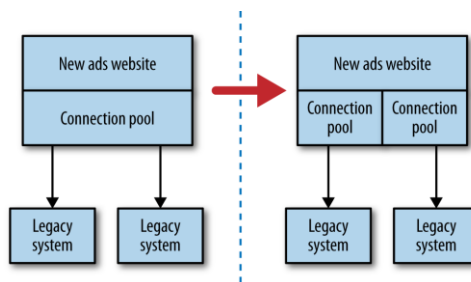
- **Alarms (SLO-oriented):**

- Lag/backlog above threshold for N minutes, DLQ depth > 0 (sustained), retry rate spikes, zero-consumption (stuck consumer), schema validation failures.

Architectural Safety Measures

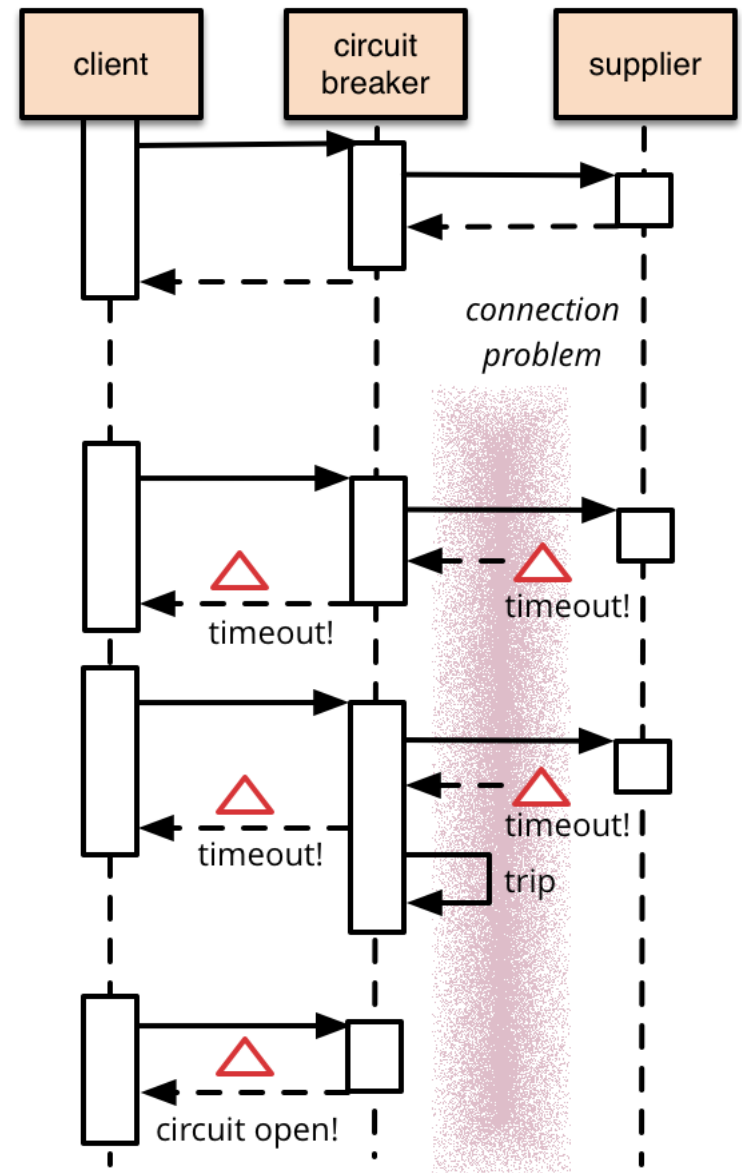
Antifragile Organization *Embracing Failure to Improve Resilience and Maximize Availability* (Chaos Monkey, et al.)

- Timeouts
- Circuit Breaker
- Bulkhead
- Isolation (microservice)



Circuit Breaker Pattern

- Remote calls can fail, or hang without a response until some timeout limit is reached
- If you have many callers on a unresponsive supplier, then you can run out of critical resources leading to cascading failures across multiple systems
- In his excellent book [Release It](#), Michael Nygard popularized the Circuit Breaker pattern to prevent this kind of catastrophic cascade
- Basic Idea:
 - Wrap a protected function call in a circuit breaker object, which monitors for failures
 - Once the failures reach a certain threshold, the circuit breaker trips, and all further calls to the circuit breaker return with an error, without the protected call being made at all
 - Usually you'll also want some kind of monitor / alert if the circuit breaker trips
 - Reset mechanism, for once the call starts working again

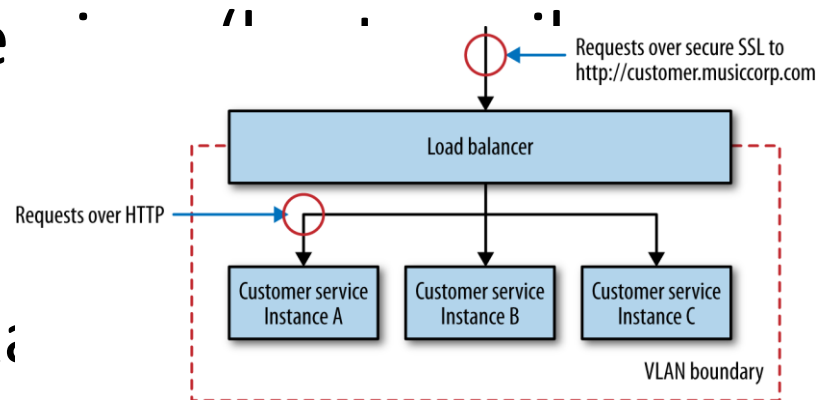


Idempotency

- From a RESTful service standpoint, for an operation (or service call) to be **idempotent**, clients can make that same call repeatedly while producing the same result.
- Idempotent is not the same as stateless: the server may keep state, but repeating the call leaves that state exactly as one call left it. GET, PUT and DELETE are defined idempotent; POST is not, unless you give it an idempotency key
- NOTE: Some of the HTTP verbs, such as GET and PUT, are defined in the **HTTP specification** to be idempotent, but for that to be the case, they rely on the service handling these calls to do so in an

Scaling

- Vertical Scaling (bigger machines)
- Horizontal Scaling (many small machines)
- Current practice: containers on a managed orchestrator (Kubernetes, ECS, Cloud Run), with serverless for spiky or event-driven pieces
- Affinity and Anti-affinity of services (zone, region)
- Load Balancer
- Worker based (Spark, Flink, Raft)



Idempotency and Scaling

- **Scaling with Replication:** In a distributed system with multiple instances of a web service, requests may be routed to different nodes for load balancing or fault tolerance. If an operation is idempotent, it doesn't matter which node processes the request because the outcome will be the same.
- **Avoiding Unintended Side Effects:** When scaling web services, it's crucial to prevent unintended side effects caused by duplicate or retried requests. Idempotency helps by ensuring that if a request is repeated due to network issues, client retries, or other reasons, it won't produce unintended consequences.
- **Caching and Optimization:** Idempotent operations can be cached more effectively. When a web service scales, caching becomes crucial to reduce load on backend systems and improve response times.
- **Simplified Error Handling:** When scaling web services, there is a higher likelihood of transient errors, such as network timeouts or service unavailability. Idempotent operations simplify error handling by allowing clients to retry requests without fear of causing inconsistencies.

Types of load-balancers

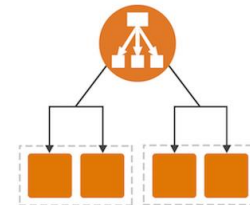
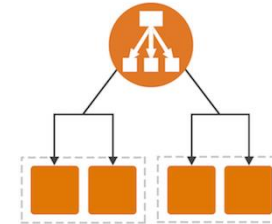
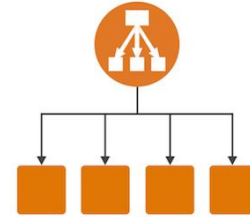
- **Classic Load Balancer (legacy; new designs use ALB or NLB)**
- **This load balancer is usually abbreviated ELB** for Elastic Load Balancer, as this was its name when it was first introduced in 2009 and was the only type of load balancer available
 - It can be thought of as an Nginx or HAProxy instance
- In 2016, AWS launched its **Elastic Load Balancing version 2**, which is made up of two offers:

Network Load Balancer (NLB)

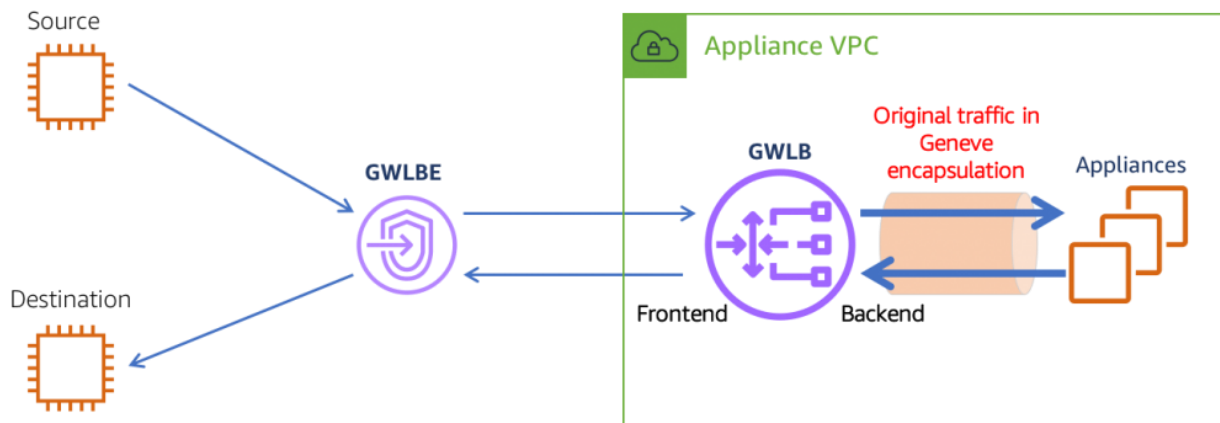
- Uses the concept of “target groups,” which is one additional level of redirection
- It can be conceptualized in this way. Listeners receive requests and decide (based on a wide range of rules) to which target group they will forward the requests
- A target group then routes the requests to instances, containers, or IP addresses. Target groups manage the targets in terms of deciding how to split up the traffic and by performing health checks on the targets
- Both ALB and NLB can forward traffic to IP addresses, which allows them to have targets outside the AWS Cloud

Application Load Balancer (ALB)

- (ALB) only works at layer 7 (HTTP)
- Has a wide range of routing rules for incoming requests based on host name, path, query string parameter, HTTP method, HTTP headers, source IP, or port number
- In contrast, ELB only allows routing based on port number. Also, contrary to ELB, ALB can route requests to many ports on a single target. Plus, ALB can route requests to Lambda functions.



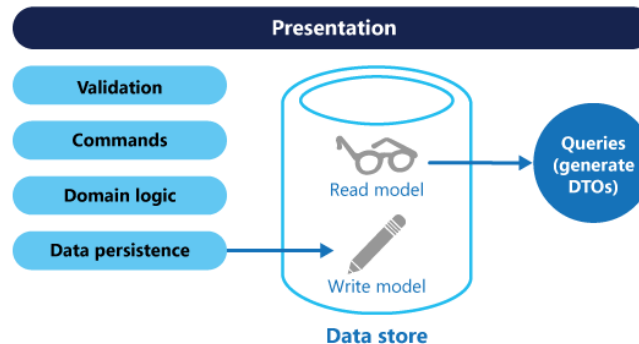
Gateway Load Balancer (GWLB)



- **Purpose:** Deploy/scale inline network virtual appliances (NGFW, IDS/IPS, DPI).
- **How it works:** Layer 3 service; encapsulates traffic with [GENEVE](#) over UDP 6081 to appliance targets.
- **Insertion:** Use GWLB Endpoints (GWLBE, via [AWS PrivateLink](#)) from consumer VPCs; expose an Endpoint Service backed by GWLB in the provider VPC.
- **Targets & health:** Target groups can be Instances or IPs; per-target-group health checks.
- **Flow handling:** Hash-based flow stickiness for stateful appliances; use TGW Appliance Mode to ensure symmetric return when centralizing.
- **Common patterns:** Centralized inspection VPC, per-VPC insertion, north–south and east–west inspection.

Scaling Databases

- Scaling for Reads (Caching & Read Replicas)
- Scaling for Writes (Sharding – data spread over many nodes) To write, a hashing function based on primary key determines destination node) NOTE, in Cassandra data resiliency is ensured by replicating to multiple nodes in a *ring*
- CQRS (Command-Query Responsibility Segregation) - an alternate model for storing and querying information. At its heart is the notion that you can use a different model to update information than the model you use to read information. *[NOTE: Can be difficult to implement correctly]*
 - CQRS separates reads and writes into different models, using commands to update data, and queries to read data.
 - Commands should be task-based, rather than data centric. ("Book hotel room", not "set ReservationStatus to Reserved").
 - Commands may be placed on a queue for asynchronous processing, rather than being processed synchronously.
 - Queries never modify the database. A query returns a DTO that does not encapsulate any domain knowledge.



Caching

- Client-Side Caching - client stores the cached result. The client gets to decide when (and if) it goes and retrieves a fresh copy [possible to hint]
- Proxy Caching - proxy placed between the client and the server. *E.g. reverse proxy or content delivery network (CDN)*
- Server-Side Caching - server handles caching responsibility, e.g. [Memcached, Redis, AWS ElastiCache](#)

Service Discovery

“How does Service A talk to Service B without having any knowledge about where to find Service B?”

- DNS alone – not suited to the churn of microservice environments unless something keeps it current, which is exactly what Kubernetes does (CoreDNS + Services)
 - Ephemeral topology
 - clients often not honoring TTL values
 - failure detection, etc.
- Dynamic Service Registries
 - [Zookeeper](#)
 - Consul
 - [Eureka](#)
 - Custom
 - Manual

Containers and orchestration, in one slide

- A container is a process with its own filesystem image and resource limits (namespaces, cgroups). The image is the unit you build, test and ship.
- An orchestrator (Kubernetes; ECS; Cloud Run; Nomad) places containers on machines, restarts them, scales replicas, routes traffic to them and rolls versions out and back.
- Kubernetes objects to know: Pod, Deployment, Service, Ingress, ConfigMap and Secret, Namespace. You declare the desired state in YAML; controllers reconcile the cluster to it.
- Managed control planes (EKS, GKE, AKS) took the hardest part. What stays yours: images, resource requests and limits, health probes, autoscaling, cost.
- Sizing rule from every shared platform: request what you use. Over-requesting is paid for whether or not it runs, and on a shared cluster it is a policy violation, not headroom.

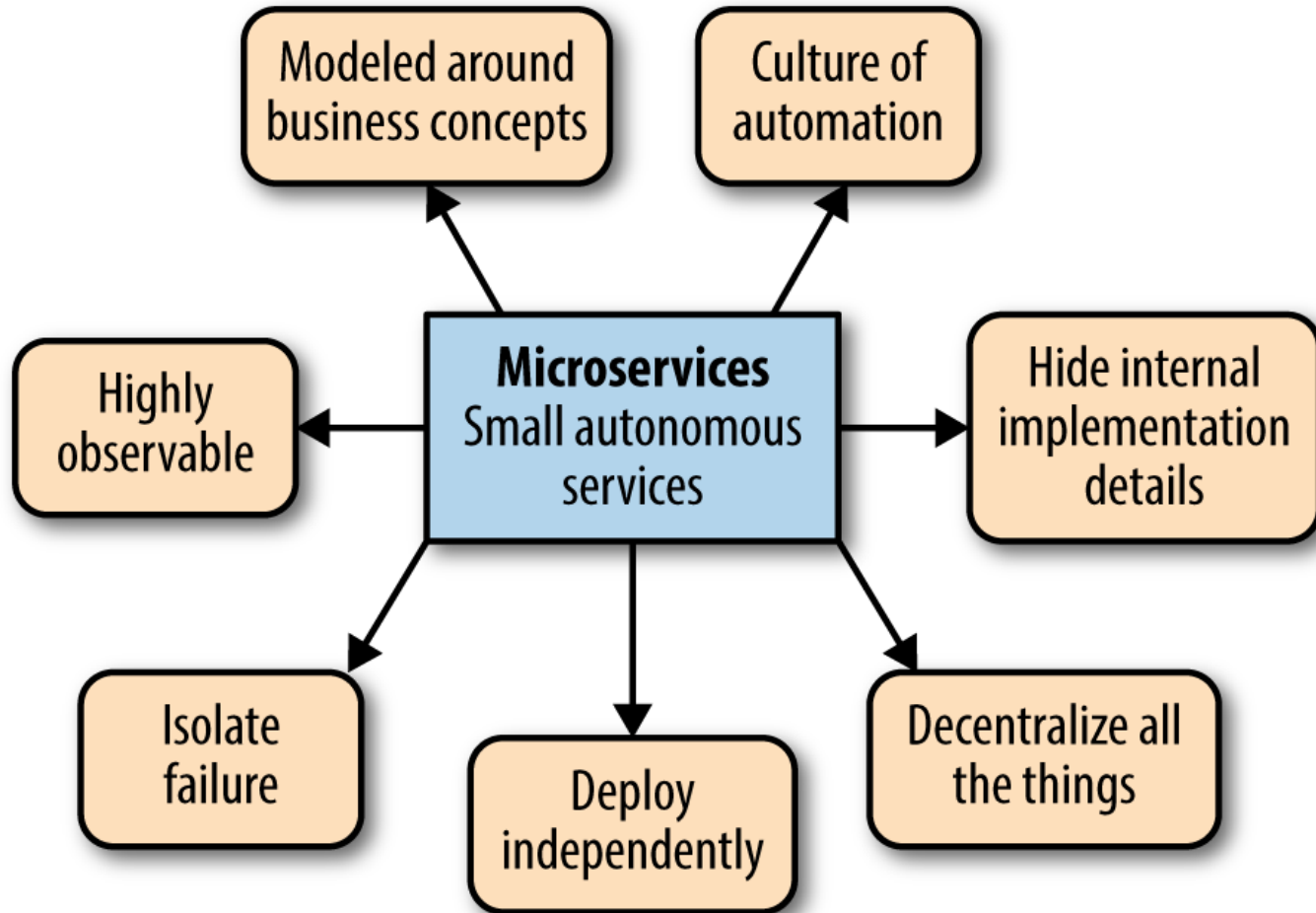
Service mesh: what it does

- A proxy beside every service (a sidecar, or a per-node proxy) intercepts every service-to-service call. Envoy and Linkerd are the proxies; Istio, Linkerd and Consul are the meshes. Application code is unchanged.
- Traffic policy in configuration, not code: timeouts, retries with budgets, circuit breaking and outlier ejection, canary and traffic splitting, fault injection for game days.
- Security by default: mutual TLS between every pair of services, an identity per workload, authorization policy at the proxy.
- Observability for free: uniform latency, error and throughput metrics, distributed traces and access logs for every hop, without instrumenting each service.
- Since 2024 the sidecar is optional: ambient or sidecar-less modes (Istio ambient, Cilium) put the proxy at the node and cut the per-pod cost.

Service mesh: where the complexity is and is not repaid

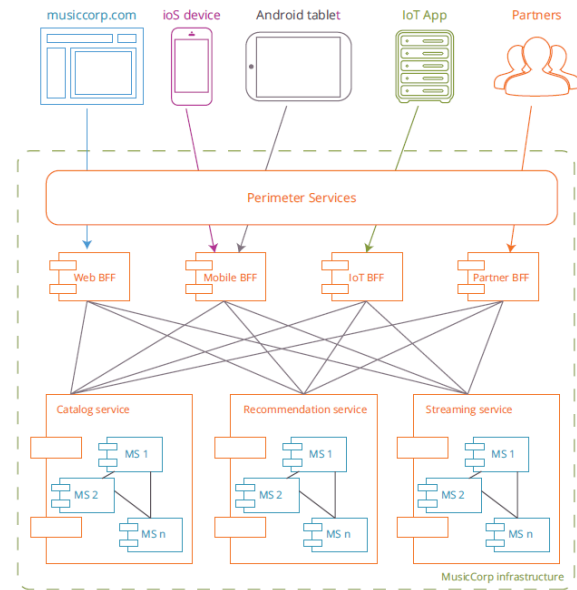
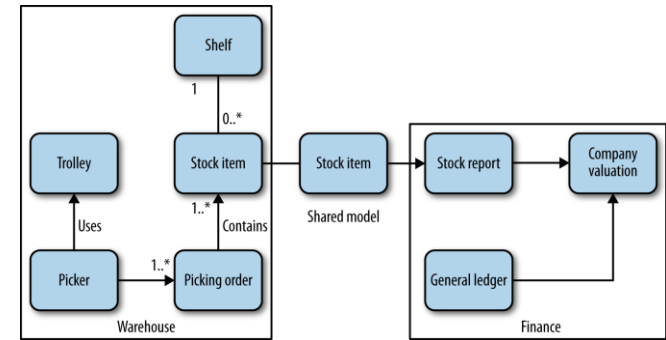
- Repaid: dozens of services in several languages; a mutual-TLS or audit requirement; timeouts and retries you cannot trust every team to set; progressive delivery across teams; a platform team that will own the mesh as a product.
- Not repaid: a handful of services in one language, where a library (Resilience4j, Polly) does the same in-process; a monolith; a team small enough that the mesh becomes the largest thing it operates.
- The costs are real: a proxy per pod (CPU and memory, about a millisecond per hop), a control plane to upgrade, a second place where policy lives, and debugging through two layers when something is slow.
- Rule of thumb: start with an API gateway at the edge and resilience libraries inside; revisit at twenty-plus services, a second language, or a compliance driver.
- The honest framing: a mesh moves resilience policy from code into configuration. That is a win only when the configuration has an owner.

Recap: Principles of Microservices



Example: MusicCorp

- *From Building Microservices*
- **Key Concepts:**
 - Loose Coupling (minimal dependencies)
 - High Cohesion (related services grouped together, unrelated services grouped separately)
 - Module boundaries
 - Business boundaries
 - Technical boundaries
 - Avoid Breaking Changes
 - Keep APIs Technology-Agnostic



<https://www.oreilly.com/library/view/building-microservices/9781491950340/ch04.html>

SOME OTHER PROBLEMS WITH MICROSERVICE ARCHITECTURES

The Inverse Conway's Law

- *Conway's Law* (named after programmer Melvin Conway in 1968):

That the architecture of a system will be determined by the communication and organizational structures of the company

- Microservice architecture is comprised of a large number of small, isolated, independent microservices. The Inverse Conway's Law demands:

That the organizational structure of any company using microservice architecture will be made up of a large number of very small, isolated, and independent teams

Technical Sprawl

- Consider a large microservice ecosystem, one containing 1,000 microservices
- Suppose each of these microservices is staffed by a development team of six developers, and each developer uses their own set of favorite tools, favorite libraries, and works in their own favorite languages
- Each of these development teams has their own way of deploying, their own specified metrics to monitor and alert on, their own external libraries and internal dependencies they use, custom scripts to run on production machines, and so on

SERVERLESS ARCHITECTURES

What is Serverless?

a cloud-native platform

for

short-running, stateless computation

and

event-driven applications

which

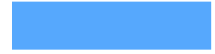
scales up and down instantly and automatically

and

charges for actual usage at a millisecond granularity

Serverless means no servers? Or worry-less about servers?

Runs code **only** on-demand on a
per-request basis



Serverless
deployment &
operations model



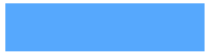
No servers



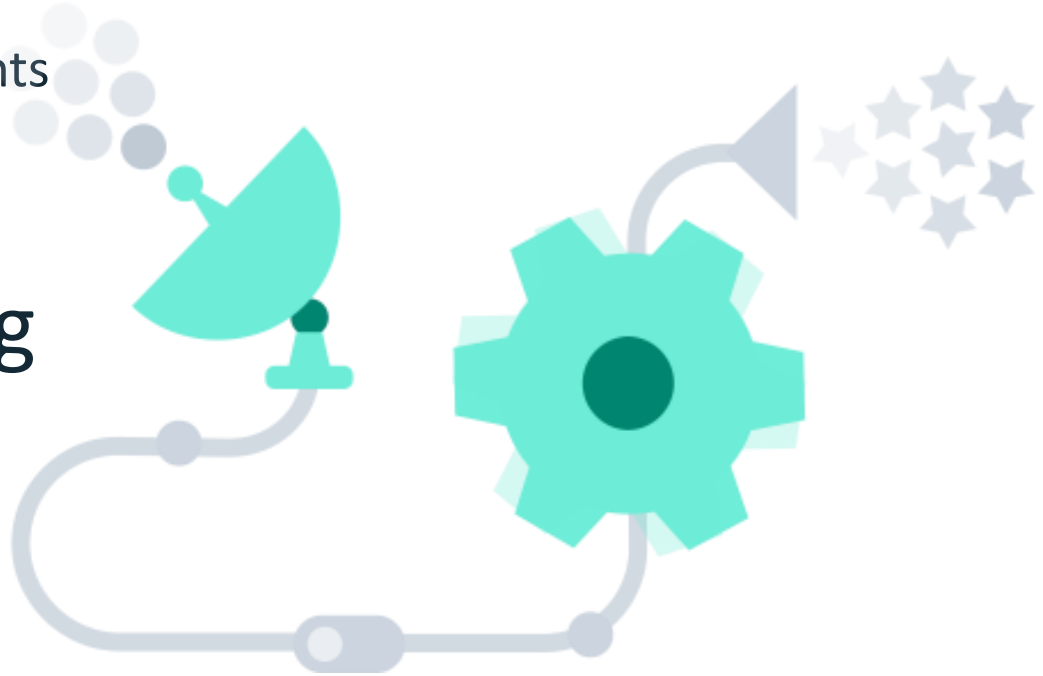
Just code

What triggers code execution?

Runs code **in response** to events



Event-programming
model



Why is Serverless attractive?

- Making app development & ops dramatically faster, cheaper, easier
- Drives infrastructure cost savings

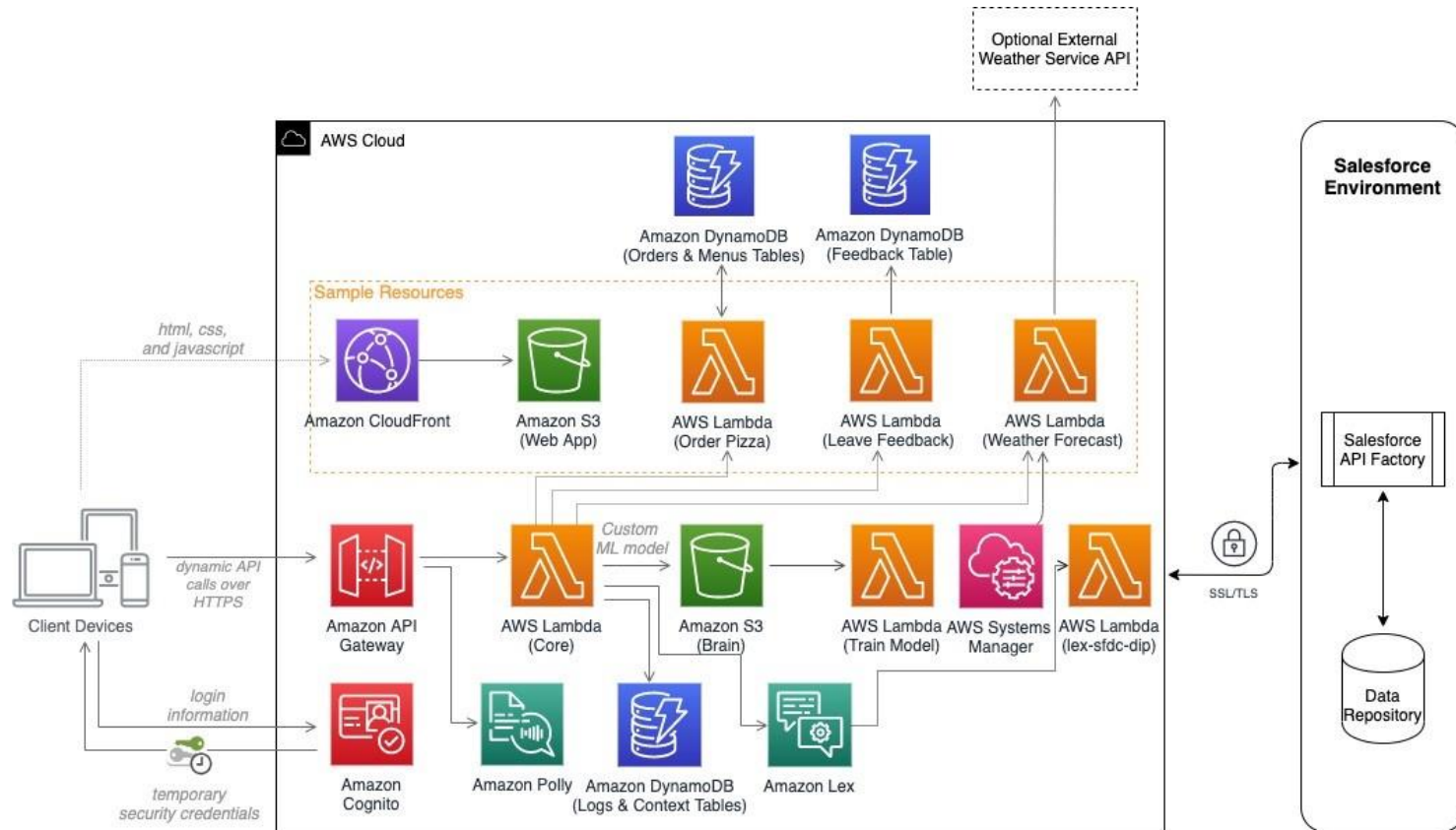
	On-prem	VMs	Containers	Serverless
Time to provision	Weeks-months	Minutes	Seconds-Minutes	Milliseconds
Utilization	Low	High	Higher	Highest
Charging granularity	CapEx	Hours	Minutes	Blocks of milliseconds

Source: Jason McGee, IBM; Serverless Conference 2017.

Key factors for infrastructure cost savings

	Traditional models (CF, containers, VMs)	Serverless
High Availability	At least 2-3 instances of everything	No incremental infrastructure
Multi-region deployment	One deployment per region	No incremental infrastructure
Cover delta between short (<10s) load spikes and valleys (vs average)	~2x of average load	No incremental infrastructure
Example incremental costs	2 instances x 2 regions x 2 = 8x	1x

Example: Serverless Chatbot



Notional architecture for Serverless Bot Framework & Salesforce Integration

<https://aws.amazon.com/blogs/architecture/build-chatbots-using-serverless-bot-framework-with-salesforce-integration/>

What is Serverless good for?

Serverless is **good** for
short-running
stateless
event-driven



Microservices



Mobile Backends



Bots, ML Inferencing



IoT



Modest Stream Processing



Service integration

Serverless is **not good** for
long-running
stateful
number crunching



Databases



Deep Learning Training



Heavy-Duty Stream Analytics



Numerical Simulation



Video Streaming

Current Platforms for Serverless



Azure
Functions



Red-Hat



AWS
Lambda



Google
Functions



OpenLambda



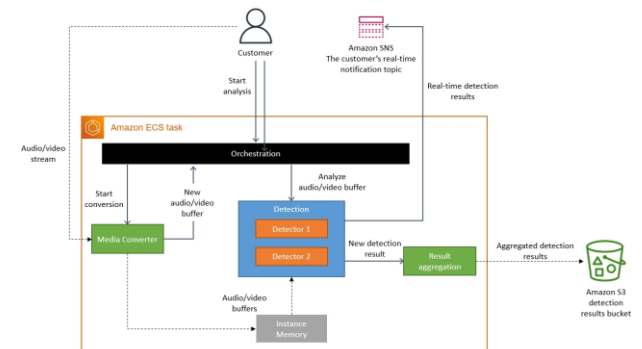
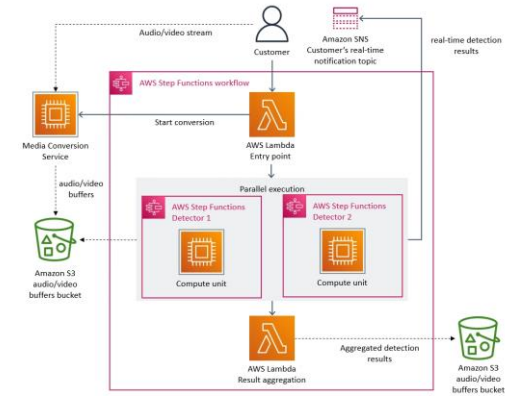
Kubernetes



AWS Lambda: https://www.youtube.com/watch?v=eOBq_h4OJ4 (3 mins)

Caveat Emptor (Prime Video, 2023)

- [Prime Video reduces costs by 90% by switching from distributed microservices to a monolith application](https://www.networkworld.com/article/3691629/cloud-vs-on-prem-saas-vendor-37-signals-bails-out-of-the-public-cloud.html)
- <https://www.networkworld.com/article/3691629/cloud-vs-on-prem-saas-vendor-37-signals-bails-out-of-the-public-cloud.html>
- <https://www.networkworld.com/article/3697737/6-lessons-from-the-amazon-prime-video-serverless-vs-monolith-flap.html>



In-class Exercise (Team)



Part A

- Create your first Lambda function

1. [Prerequisites](#)
2. [Create the function](#)
3. [Invoke the function](#)
4. [Clean up](#)
5. [Next steps](#)

Part B

- Choose one of the following tutorials for more complex examples of using Lambda with other AWS services.

1. [Using Lambda with API Gateway](#): Create an Amazon API Gateway REST API that invokes a Lambda function.
2. [Using a Lambda function to access an Amazon RDS database](#): Use a Lambda function to write data to an Amazon Relational Database Service (Amazon RDS) database through RDS Proxy.
3. [Using an Amazon S3 trigger to create thumbnail images](#): Use a Lambda function to create a thumbnail every time an image file is uploaded to an Amazon S3 bucket.



HOMEWORK

HW #3 – Serverless Patterns

- Serverless applications are built using many services in a few common architecture patterns. Although customer scenarios are unique, the patterns emerge again and again.
- <https://catalog.workshops.aws/serverless-patterns/en-US>
- In this workshop, you will learn serverless best practices by building production-ready code for an application.
 - Module 1: Intro to Serverless — Start here!
 - Module 2: Synchronous Invocation
 - Module 3: Synchronous + Idempotence
 - Module 4: Asynchronous Invocation
 - Module 5: Long running task status via long polling
- Due before your section's next session: sec 49 by Wed Sep 16, sec 03 by Tue Sep 22. Disclose AI-tool use.

References

- <https://martinfowler.com/microservices/>
- <https://martinfowler.com/articles/microservice-trade-offs.html>
- <https://www.redhat.com/cms/managed-files/mi-3scale-achieving-enterprise-microservices-ebook-f7916-201706-en.pdf>
- <https://martinfowler.com/bliki/MonolithFirst.html>
- <https://dzone.com/articles/patterns-for-microservices-sync-vs-async>
- Occupy the Cloud: Distributed Computing for the 99%, Eric Jonas, Qifan Pu, Shivaram Venkataraman, Ion Stoica, Benjamin Recht, <https://arxiv.org/abs/1702.04024>
- Building a Chatbot with Serverless Computing, Yan, Mengting and Castro, Paul and Cheng, Perry and Ishakian, Vatche, Proceedings of the 1st International Workshop on Mashups of Things and APIs 2016
- [11th IEEE/ACM International Conference on Utility and Cloud Computing \(UCC\)](#) and [5th IEEE/ACM International Conference on Big Data Computing, Applications and Technologies \(BDCAT\)](#)

SAN JOSÉ STATE UNIVERSITY *powering* SILICON VALLEY